



**Dpto. Sistemas Informáticos y Computación
Escuela Técnica Superior de Ingeniería Informática
UNIVERSITAT POLITÈCNICA DE VALÈNCIA**

Técnicas, Entornos y Aplicaciones De Inteligencia Artificial

Fuzzy-CLIPS

1.- Variables difusas. Valores difusos, funciones de pertenencia. Modificadores.	2
1.1.- Modificadores Lingüísticos	3
2.- Hechos Difusos.	4
2.1.- Declaración de hechos iniciales: Deffacts	4
2.2.- Declaración de hechos difusos mediante expresiones ASSERT.	4
2.3.- Fusificación de valores Crisp	6
2.4.- Lectura de valores Crisp o valores difusos por consola.....	6
3.- Reglas difusas.	7
4.- Inferencia difusa.	9
5.- Defusificación.	10
Anexo. Hechos estructurados. Templates.	12

FuzzyCLIPS

FuzzyClips es la extensión fuzzy de CLIPS para manipular hechos y reglas difusas, ofreciendo el soporte para el desarrollo de sistemas difusos en el entorno de CLIPS. FuzzyClips es de dominio público (desarrollado por el Inst. for Information Technology, National Research Council of Canada) y permite integrar datos y reglas difusas (imprecisas) y no difusas. FuzzyClips puede descargarse de Poliformat.

La **instalación de FuzzyClips** consiste simplemente en copiar en un mismo directorio los componentes: **FuzzyClips.exe** y **CLIPSedt.exe**. El presente documento es una simplificación de las funcionalidades presentes en FuzzyClips, que puede extenderse con la información en su manual.

1.- Variables difusas. Valores difusos, funciones de pertenencia. Modificadores.

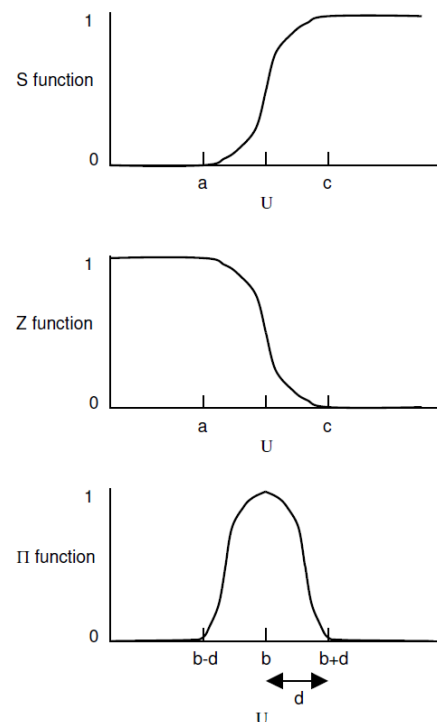
Las **variables difusas** se crean a partir de plantillas (**templates**), donde también se define su **Universo**, sus **valores difusos** (o valores lingüísticos) mediante las correspondientes **funciones de pertenencia**.

En la plantilla se define:

- La **variable difusa**, por ej. edad, temperatura, etc.
- **Universo** (límites del universo de valores)
- Los **valores difusos** que puede tomar, por ej. alto, medio, bajo, etc.
- Las **funciones de pertenencia** asociados a cada uno de los valores difusos.

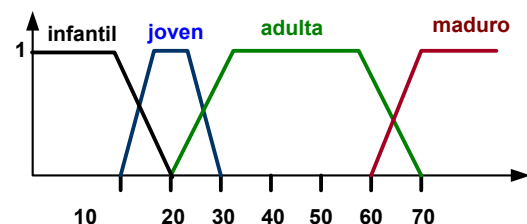
Las funciones de pertenencia se pueden definir de dos maneras diferentes:

- a) Definiendo los puntos característicos (función de pertenencia específica). Pueden indicarse tantos puntos como sea necesario. El último valor se mantiene hasta el límite superior del universo.
- b) Usando unas funciones ya predefinidas: s, z, pi (Más información en el manual).



Ejemplos:

```
(deftemplate edad ;Variable difusa
  0 120 años ;Universo
  ( (infantil (12 1) (20 0)) ;Valores difusos
    (joven (10 0) (15 1) (25 1) (30 0))
    (adulta (20 0) (30 1) (60 1) (70 0))
    (mayor (60 0) (70 1))))
```



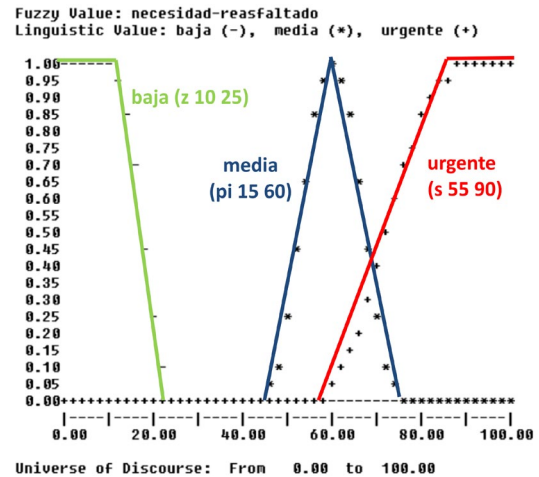
```
(deftemplate singleton
  0 10 unit
  ( (tres (3 0) (3 1) (3 0))
    (cinco (5 0) (5 1) (5 0))))
```



```
(deftemplate estatura 0 250 cm
  ( (bajo (0 1) (100 1) (150 0))
    (medio (100 0) (150 1) (170 1) (180 0))
    (alto (170 0) (180 1))))
```

```
(deftemplate temperatura 30 50 grados
  ( (bajo (35 1) (37 0))
    (medio (35 0) (36 1) (37 0))
    (alto (36 0) (37 1))))
```

```
(deftemplate necesidad-reasfaltado 0 100 unidades
  ( (baja (z 10 25))
    (media (pi 15 60))
    (urgente (s 55 90))))
```



1.1.- Modificadores Lingüísticos

Los modificadores lingüísticos modifican la forma de un conjunto difuso previamente definido. Por ejemplo:

$$\text{very}(y) = y^2 \qquad \text{more-or-less}(y) = \sqrt{y}$$

FuzzyClips provee los siguientes principales modificadores lingüísticos:

not	$1-y$	very	y^2	somewhat	$y^{1/3}$
more-or-less	$\sqrt{y}$	extremely	y^3	plus	$y^{1.25}$

Por ejemplo, podemos tener:

```
(deftemplate estatura 0 250 cm
  ( (bajo (0 1) (100 1) (150 0))
    (muybajito extremely bajo)
    (medio (100 0) (150 1) (170 1) (180 0))
    (alto (170 0) (180 1))
    (muy-alto very alto)))
```

```
(deftemplate temperatura 0 80 grados
  ( (frio (z 10 25))
    (congelado plus frio)))
```

Los operadores lingüísticos también se pueden combinar con expresiones lógicas:

```
(deftemplate temperatura 0 80 grados
  ((frio (z 10 25))
   (calor (s 30 40))
   (templado not [ frio OR calor ]))) ;hay que dejar un espacio después/antes de "[ ]"!!
```

2.- Hechos Difusos.

Un **hecho difuso** es una expresión de la forma:

(variable-difusa valor-difuso),
(variable-difusa modificador valor-difuso), o
(variable-difusa expresión-lógica-difusa),

Donde 'variable-difusa' debe haber sido definida previamente e incluir, entre sus valores, el valor-difuso que se aserta.

Las expresiones lógicas combinan valores difusos mediante expresiones **AND** (mínimo de los valores de las funciones de pertenencia), **OR** (máximo de los valores de las funciones de pertenencia) y **NOT** (1- función de pertenencia). **NOTA:** el operador AND tiene más prioridad que el OR.

Ejemplo

Una vez definidas las variables difusas 'edad' y 'estatura' anteriores, podemos tener como hechos difusos:

(edad adulta), (estatura very bajo), (edad adulta OR joven), (edad adulta AND joven), etc.

- a) Estos hechos pueden ser asertados mediante expresiones assert:

(assert (edad adulta)) (assert (estatura very bajo)) (assert (edad adulta OR joven))

- b) O inicializados mediante expresiones deffacts:

(deffacts ejemplo
 (edad adulta) (estatura very bajo))

2.1.- Declaración de hechos iniciales: Deffacts

La expresión **deffacts** permiten declarar un conjunto de hechos iniciales crisp y fuzzy que se inicializarían con '(reset)'. Por ejemplo:

(deffacts ejemplo1
 (edad adulta)
 (estatura very alto))

Podemos, posteriormente, añadir nuevos hechos mediante expresiones assert.

2.2.- Declaración de hechos difusos mediante expresiones ASSERT.

Se pueden asertar nuevos hechos difusos mediante expresiones **assert**:

(assert (<crisp-fact> |
 fuzzy-variable-name <description of fuzzy set>))

donde <description of fuzzy set> puede ser un <valor difuso> definido para la variable, o bien el par <modificador> <valor difuso>, o bien una expresión lógica difusa:

<description of fuzzy set> := <valor difuso> | < modificador> <valor difuso> | <fuzzy-logical-expresion>

Ejemplos:

Supongamos definidas las variables y sus valores difusos:

(deftemplate edad 0 100 años ((joven (10 0) (15 1) (25 1) (30 0)) (adulta (20 0) (30 1) (60 1) (70 0)) (madura (60 0) (70 1))))	(deftemplate estatura 0 250 cm ((bajo (0 1) (100 1) (150 0)) (medio (100 0) (150 1) (170 1) (180 0)) (alto (170 0) (180 1))))
--	---

Podemos declarar **hechos difusos** de la forma:

```
(assert (edad adulta))  
(assert (edad joven OR adulta OR madura)) ;se podrían plantear varias edades  
(assert (edad joven AND adulta)) ;se podrían plantear varias edades  
(assert (estatura very alto))
```

Nota: si asertamos de forma independiente (assert (edad adulta)) y luego (assert (edad joven)), no estaremos asertando el equivalente a (assert (edad adulta AND joven)), sino (assert (edad adulta OR joven))

Asimismo, asumida definida la variable difusa:

```
(deftemplate grupo 0 20 miembros ;declaración de la variable grupo  
((pocos (3 1) (6 0)) ;valor pocos  
(muchos (4 0) (6 1)))) ;valor muchos
```

Podemos asertar hechos como:

```
(assert (grupo pocos))  
(assert (grupo (1 0) (5 1) (7 0)) ) ;Nuevo valor  
(assert (grupo NOT [ very pocos OR muchos ] )) ;es necesario dejar un espacio junto a [ y ]  
(assert (grupo (z 4 8)))
```

IMPORTANTE

No se deben declarar o asertar hechos **no estructurados** que contengan hechos difusos o mezclados con hechos Crisp.

Por ejemplo, si asertamos:

```
(juan edad adulta)  
(juan edad adulta estatura bajo)  
(persona nombre adan edad adulta estatura alto)
```

Esta información la asumiría como información Crisp, es decir, no difusa.

2.3.- Fusificación de valores Crisp

Hemos visto cómo se puede asertar información difusa mediante expresiones `assert`. Pero hay casos en los que la información de partida es un valor Crisp, es decir un valor concreto no difuso.

Por ejemplo, imaginemos que sabemos que la edad es de 35 años y la altura de 172 cm y queremos asertar dicha información a variables difusas. Para ello podemos utilizar la siguiente función *fuzzify* que viene como ejemplo con el manual de usuario de FuzzyClips (ejemplo llamado `fuzzify.clp`):

```
(deffunction fuzzify (?fztemplate ?value ?delta)

  (bind ?low (get-u-from ?fztemplate))
  (bind ?hi (get-u-to ?fztemplate))

  (if (<= ?value ?low)
    then
      (assert-string
        (format nil "(%s (%g 1.0) (%g 0.0))" ?fztemplate ?low ?delta))
    else
      (if (>= ?value ?hi)
        then
          (assert-string
            (format nil "(%s (%g 0.0) (%g 1.0))"
              ?fztemplate (- ?hi ?delta) ?hi))
        else
          (assert-string
            (format nil "(%s (%g 0.0) (%g 1.0) (%g 0.0))"
              ?fztemplate (max ?low (- ?value ?delta))
              ?value (min ?hi (+ ?value ?delta)) ))
          )))
```

Esta función se puede invocar de la forma *(fuzzify edad-difusa 35 0.1)* y asertará el valor fusificado en la variable difusa *edad* mediante la expresión:

```
(assert-string "(edad-difusa (34.9 0) (35 1) (35.1 0))").
```

El rango de la función de pertenencia del valor asertado se establece en la llamada a la función. Particularmente, si queremos definir un valor *singleton*, será tan sencillo como invocar “(fuzzify edad 35 0)”.

Nota: Fuzzy-clips puede dar problemas si el valor crisp coincide con alguno de los extremos del intervalo definido para la variable difusa.

2.4.- Lectura de valores Crisp o valores difusos por consola

A menudo es útil introducir al sistema hechos que correspondan a valores leídos por consola, mediante la expresión `(read)`, y asertar dichos valores como hechos del sistema. Esto permite una mayor interacción con el usuario.

Sin embargo, fuzzy-clips **no permite asertar valores difusos que sean leídos directamente desde consola**. Los valores difusos necesitan ser explícitamente asertados mediante expresiones `assert` o `deffacts`.

Por ejemplo, si tenemos definidas las variables difusas *edad* y *estatura*:

```
(deftemplate edad 0 100 años
  (joven (10 0) (15 1) (25 1) (30 0))
  (adulto (20 0) (30 1) (60 1) (70 0))
  (madura (60 0) (70 1)))

(deftemplate estatura 0 250 cm
  ((bajo (0 1) (100 1) (150 0))
   (medio (100 0) (150 1) (170 1) (180 0))
   (alto (170 0) (180 1))))
```

Podríamos asertar desde consola: `(assert (edad adulto))`. Pero no podríamos asertar un valor ‘adulto’ leyendo dicho valor en consola. Para poder introducir valores difusos, a partir de valores crisp o de símbolos (por ejemplo: alto, bajos, etc.) leídos por consola podemos utilizar la función `assert-string`.

➤ **Lectura de símbolos (como valores fuzzy):**

Podemos utilizar la siguiente regla para asertar como valor difuso el mismo símbolo (que representa un valor fuzzy) leído por consola.

```
(defrule leerconsola ;lee de consola un valor difuso y lo aserta
  (initial-fact)
=>
  (printout t "Introduzca la edad: joven, adulta, madura" crlf)
  (bind ?Redad (read))
  (assert-string (format nil "(edad %s)" ?Redad)) )
```

Esta regla también podría codificarse como una función para lectura de valores y su aserción fuzzy.

➤ **Lectura de valor crisp y aserción de su valor fusificado:**

Podemos usar la función **fuzzify** para asertar un valor difuso en función de un valor crisp leído:

```
(defrule leerconsola ;lee de consola un valor crisp y aserta su valor fusificado
  (initial-fact)
=>
  (printout t "Introduzca la edad en anyos" crlf)
  (bind ?Redad (read))
  (fuzzify edad ?Redad 0.1))
```

Y también, en caso de querer asertar la fusificación *singleton* de un valor crisp:

```
(defrule leerconsola ;lee de consola valor crisp y aserta su valor fuzzy singleton
  (initial-fact)
=>
  (printout t "Introduzca la edad en anyos" crlf)
  (bind ?Redad (read))
  (assert (edad (?Redad 0) (?Redad 1) (?Redad 0))))
```

3.- Reglas difusas.

Antecedentes

Los antecedentes de una regla difusa representan condiciones sobre hechos difusos o hechos crisp. Las condiciones fuzzy son de la forma:

(fuzzy-variable-name <linguistic-expr>) | (fuzzy-variable-name ?<var-name>) | (fuzzy-variable-name ?) | (fuzzy-variable-name ?<var-name> & <linguistic-expr>)

Donde <linguistic-expr> puede representar un valor difuso, con modificadores y/o expresiones lógicas (and/or).

Además, se admite el conjunto de patrones sobre valores crisp y condiciones <test>.

Consecuentes

El consecuente de una regla difusa utiliza las expresiones assert previamente vistas, así como el resto de funciones Clips.

EJEMPLO-1:

Supongamos la siguiente declaración de variable difusa:

```
(deftemplate tanque 0 80 litros
  ((bajo (10 1)(30 0))
   (medio (20 0)(35 1)(45 1)(60 0))
   (alto (50 0)(70 1))))
```

Y asertado el hecho simple: (assert (tanque plus alto))

```
Podemos escribir una regla: (defrule danger
                             (tanque extremely alto)
                             =>
                             (printout t "El tanque puede desbordarse. PELIGRO!" crlf)
                             (assert (alarma)))
```

Después de hacer “(reset)” y “(run)” se obtendría la conclusión alarma (CF=0.85), en la medida en que alto cumple (0.85) la condición de extremadamente alto.

Notas importantes:

- No es posible utilizar en una regla la función ‘test’ con valores difusos. Es decir:

```
(defrule example
  (tanque ?h))
  (test (eq ?h alto))
```

no haría matching difuso.

- La función (get-u-units ?h) devuelve un *string* que corresponde a las unidades en las que se ha definido el valor difuso ?h

También:

<i>Función</i>	<i>Devuelve</i>
(get-u-units <fact-index>)	Unidades del conjunto
(get-u-from <fact-index>)	Valor inferior universo
(get-u-to <fact-index>)	Valor superior universo
(get-u <fact-index>)	Rango del universo

- Para introducir valores Crisp, y que puedan hacer matching con valores difusos, es necesario utilizar la función *fuzzify* explicada en el apartado 2.3 o definir el valor Crisp como un valor difuso (de tipo singleton en la propia definición de la variable difusa).

EJEMPLO-2:

Conocemos que una persona tiene 25 años, y tenemos definidos los conjuntos difusos:

```
(deftemplate edad 0 100 años
  ( (joven (10 0) (15 1) (25 1) (30 0))
    (veinticinco (25 0) (25 1) (25 0)) ;definimos el singleton, edad 25.
    (adulto (20 0) (30 1) (60 1) (70 0))
    (madura (60 0) (70 1))))

(deftemplate estatura 0 250 cm
  ( (bajo (0 1) (100 1) (150 0))
    (medio (100 0) (150 1) (170 1) (180 0))
    (alto (170 0) (180 1))))
```

Ahora tenemos el hecho inicial: (defacts ejemplo (edad veinticinco))

Podemos tener la regla:

```
(defrule edades
  (edad adulta)
  =>
  (printout t "Los adultos son altos" crlf)
  (assert (estatura alto)))
```

La premisa (edad adulta) hará matching con el hecho difuso asertado (edad veinticinco). Análogamente, podíamos NO haber definido el singleton veinticinco y haber usado la función "(fuzzify edad 25 0)" del apartado 2.3. En el primer caso hemos definido un valor singleton que estará siempre disponible en la variable difusa edad, mientras que en el segundo caso simplemente asertamos información Crisp que está siendo fusificada.

NOTA. Para modificar la prioridad de una regla se puede utilizar la construcción "(declare (salience X))" en la parte izquierda de la misma. El valor X es un valor entero (positivo o negativo); cuanto mayor valor, mayor es la prioridad y viceversa. Si, por ejemplo, queremos que una regla tenga la mínima prioridad podemos ponerle un valor muy negativo, ej. (declare (salience -20)).

4.- Inferencia difusa.

FuzzyClips utiliza el **modus ponens difuso** para obtener las consecuencias inferidas. Se puede elegir entre dos reglas composicionales, Max-min y Max-prod:

```
(set-fuzzy-inference-type <tipo>)
```

que son las dos reglas más utilizadas en el razonamiento difuso.

EJEMPLO: Inferencia difusa sobre el control de la temperatura de una habitación.

Primero definimos las variables difusas.

ENTRADA: TEMPERATURA.

```
(deftemplate Temp 5 50 Celsius
  ((frio (z 10 20))
   (templado (pi 5 25))
   (calor (s 30 40))))
```

SALIDA: APERTURA DE LA VALVULA

```
(deftemplate valvula 0 90 grados-apertura
  ((poco (z 10 30))
   (medio (pi 30 45))
   (mucho (s 70 80))))
```

Definimos las reglas:

```
(defrule hace_frio          (defrule temperatura_buena      (defrule hace_calor
  (Temp frio)                (Temp templado)                (Temp calor)
  =>                          =>                          =>
  (assert (valvula mucho)))  (assert (valvula medio)))  (assert (valvula poco)))
```

Tras los hechos iniciales:

```
(deffacts ejemplo
  (Temp very templado))
```

Iniciamos la base de conocimiento con "(reset)" y lanzamos la ejecución con "(run)", lo que asertará los valores correspondientes a la variable difusa válvula.

```
f-0      (initial-fact) CF 1.00
f-1      (Temp very templado) CF 1.00
f-2      (valvula medio) CF 1.00 1.25 0.01563 (21.56 0.04785) (21.88 0.09766)
```

Podríamos utilizar una regla para defusificar dicho valor, y obtener un valor numérico (un ángulo) con el que girar la válvula (ver punto sobre defusificación):

```
(defrule defuzzificar
```

```

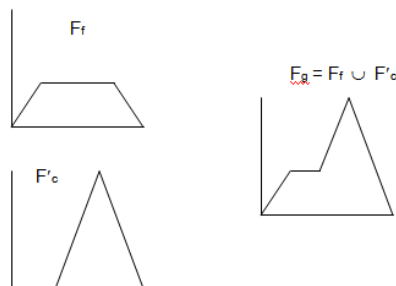
(valvula ?val)
=>
(printout t "Apertura Valvula por moment: " (moment-defuzzify ?val) crlf))
(printout t "Apertura Valvula por maximum: " (maximum-defuzzify ?val) crlf))

```

Nota sobre duplicación de hechos difusos

En Clips estándar si se aserta un hecho que ya existe no se permite duplicación de hechos. De esta forma, las reglas no se vuelven ejecutar sobre un mismo hecho. Sin embargo, en un sistema difuso, el refinamiento de un hecho difuso puede ser posible:

- a) Si se aserta un nuevo valor difuso a una variable, distinto al existente, se combinan ambos valores considerando una combinación OR ($F_g = F_f \cup F'_c$):



Por ejemplo, si asertamos: `(assert (Temp frio))`
`(assert (Temp templado))`

Se supone que se añade más información, y resulta el hecho: `(Temp [frio] OR [templado])`

- b) De esta forma, una regla previamente ejecutada sobre este hecho, volverá a ejecutarse con la nueva información.

Por ejemplo, si se aserta “`(assert (Temp frio))`” y se lanza la regla “`hace_frio`”, y posteriormente, se aserta “`(assert (Temp templado))`”, se puede ejecutar de nuevo la misma regla, con la nueva información, en la medida en que `(Temp templado)` satisfaga la premisa de la regla `(Temp frio)`.

5.- Defusificación.

Para realizar la defusificación de un conjunto difuso a un valor numérico, se dispone de dos funciones:

`(moment-defuzzify ?fuzzy-value | ?variable-fuzzy | integer)`, que aplica el algoritmo del centro de gravedad.

`(maximum-defuzzify ?fuzzy-value | ?variable-fuzzy | integer)`, que aplica la media de máximos.

El valor integer corresponde a la identificación de un valor en la base de hechos (identificado en la parte izquierda de una regla).

EJEMPLO-1:

Declaramos la variable difusa y las reglas:

```

(deftemplate edad 0 100 años
  ( (joven (10 0) (15 1) (25 1) (30 0))
    (adulta (20 0) (30 0.7) (40 1) (60 0.7) (70 0))
    (madura (60 0) (70 1))))

(defrule fuzzy1
  (edad ?f)
  => (bind ?e (maximum-defuzzify ?f))
      (printout t "En fuzzy1, edad es " ?e crlf))

(defrule fuzzy2
  (edad ?f)
  => (bind ?e (moment-defuzzify ?f))
      (printout t "En fuzzy2, edad es " ?e crlf))

```

Asertamos el hecho: (assert (edad adulta)), y tras lanzar la ejecución (run), obtenemos:

```
FuzzyClips> (run)
FuzzyClips> En fuzzy1, edad es 40.0
FuzzyClips> En fuzzy2, edad es 44.76923076923077
```

Debe notarse que, si por el contrario hubiéramos especificado: (assert (edad not adulta)), obtendríamos de acuerdo a los métodos de defusificación:

```
FuzzyClips> (reset)
FuzzyClips> (assert (edad not adulta))
FuzzyClips> (run)
FuzzyClips> En fuzzy1, edad es 47.5
FuzzyClips> En fuzzy2, edad es 52.51851851852
```

También:

```
FuzzyClips> (reset)
FuzzyClips> (assert (edad madura))
FuzzyClips> (run)
FuzzyClips> En fuzzy1, edad es 85.0
FuzzyClips> En fuzzy2, edad es 82.38095238095238
```

EJEMPLO-2:

Definamos un sistema que caracteriza la aptitud de una persona para jugar al baloncesto en función de su edad, su salud y su estatura.

```
(deftemplate edad 0 100 años ; definición de la variable fuzzy edad
  ( (joven (10 0) (15 1) (25 1) (30 0))
    (adulta (20 0) (30 1) (60 1) (70 0))
    (madura (60 0) (70 1))))

(deftemplate estatura 0 250 cm ; definición de la variable fuzzy estatura
  ((bajo (0 1) (100 1) (150 0))
    (medio (100 0) (150 1) (170 1) (180 0))
    (alto (170 0) (180 1))))

(deftemplate aptitud 0 10 unidades ; aptitud para jugar al baloncesto
  ((baja (0 1) (5 0))
    (media (3 0) (4 1) (6 1) (10 0))
    (alta (5 0) (10 1))))
```

Definimos una regla de clasificación y una para defusificar

```
(defrule ejemplo ; asignar_aptitud_jugar_baloncesto
  (edad joven)
  (estatura alto)
  =>
  (assert (aptitud alta)))

(defrule defusificar
  ?f <- (aptitud ?)
  => (bind ?e (maximum-defuzzify ?f))
      (printout t "Aptitud es " ?e crlf))
```

Ahora supongamos los siguientes hechos iniciales:

```
(defacts fuzzy-fact
  (edad adulta)
  (estatura very alto))
```

Si ejecutamos, obtendremos:

```
FuzzyClips> Aptitud es 9.1666
```

Nótese que el nuevo hecho obtenido (ventana Facts) es: (aptitud ???), que está indicando que su aptitud es un valor difuso no previamente definido. Este nuevo valor difuso (en la medida en que hacen más o menos matching las premisas) es obtenido mediante la inferencia de los datos. Su defusificación obtiene el valor 9.16.

Nótese también que FuzzyClips representa, en los hechos obtenidos, la función de pertenencia del valor difuso inferido.

NOTA. Si queremos que el proceso de razonamiento finalice tras la ejecución de una regla, basta con utilizar “(halt)” en la parte derecha de la regla. La utilización de “halt” finalizará el proceso de inferencia, aunque existan reglas por aplicar. Esto puede ser necesario para evitar bucles infinitos por la posibilidad de lanzar la misma regla un número infinito de veces.

Anexo. Hechos estructurados. Uso de ontologías y templates.

Fuzzy-Clips permite la declaración de clases o templates como hechos estructurados. Una template contiene slots, los cuales solo pueden tener valores crisp¹. Una template se define de la forma:

```
(deftemplate <nombre-template> <crisp-slot>+ *)
```

Los slots contienen valores crisp, del tipo que se indique:

```
<crisp-slot> ::= (slot <slot-name> [(type <tipo>)] [(default <valor>)] )  
donde <tipo> ::= SYMBOL | INTEGER | STRING | ...
```

Nota: si se declara de tipo entero, su valor por defecto es 0

Ejemplo: Podemos declarar el template:

```
(deftemplate persona  
  (slot nombre (type SYMBOL))  
  (slot altura (type INTEGER))  
  (slot peso (type INTEGER))  
  (slot seleccion (type SYMBOL) (default nil))  
  (slot vive (type SYMBOL)))
```

Una vez declarado el template, los hechos iniciales se suelen declarar mediante la expresión deffacts:

```
(deffacts prueba  
  (persona (nombre david) (altura 184) (vive Valencia))  
  (persona (nombre juan) (altura 164) (peso 70) (vive Sevilla))  
  (persona (nombre maria) (altura 174) (vive Madrid)))
```

O también realizar aserciones como: (assert (persona (nombre luis) (vive Valencia)))

Las reglas expresan patrones que deben instanciarse sobre los hechos. Por ejemplo, si **queremos modificar** el valor de uno o varios slots usaremos “modify” sobre la instancia, indicando el nuevo valor de los nuevos slots. **NOTA IMPORTANTE:** para modificar slots se usa “modify” (se deben modificar todos los slots **simultáneamente**), pero NO debemos asertar una nueva instancia con el valor modificado (en ese caso estaríamos creando múltiples instancias).

```
(defrule seleccion  
  ?f <- (persona (nombre ?n) (altura ?a) (vive ?v) (seleccion nil))  
  (test (> ?a 175))  
  =>  
  (printout t "seleccionado " ?n crlf)
```

¹ Realmente sí se puede definir un slot difuso, pero Fuzzy Clips necesita que se inicialice siempre a un valor (teniendo poca utilidad práctica si representa una conclusión que debe calcularse mediante inferencia).

(modify ?f (seleccion si)))

De esta forma, modificaremos el valor del slot “seleccion” de la persona que ha hecho matching y se encuentra en ?f.

Para ver más ejemplos de reglas puede verse el ejemplo sobre jarras del Tema-1.

En fuzzy clips podemos mezclar hechos estructurados (solo con slots crisp) y hechos no estructurados (crisp o difusos).

En las reglas, podemos tener patrones sobre hechos estructurados y condiciones sobre valores de slots crisp, así como condiciones sobre valores de variables difusas.

Evaluación de la Práctica

El objetivo de la práctica es aplicar el razonamiento difuso a un problema concreto utilizando FuzzyClips. Tras su desarrollo, su evaluación consistirá en rediseñar y evaluar pequeñas modificaciones del problema propuesto.

Ejercicio. Sistema *Fuzzy* para la estabilización del vuelo de drones

La estabilización del vuelo de un dron, especialmente al intentar mantener un vuelo estacionario o correctamente equilibrado, se logra mediante la combinación de sensores inerciales avanzados y técnicas de control que corrigen la posición del dron en tiempo real. El objetivo es el que el dron se mantenga lo más equilibrado posible ante potenciales ráfagas de viento imprevistas que varían la inclinación del dron.

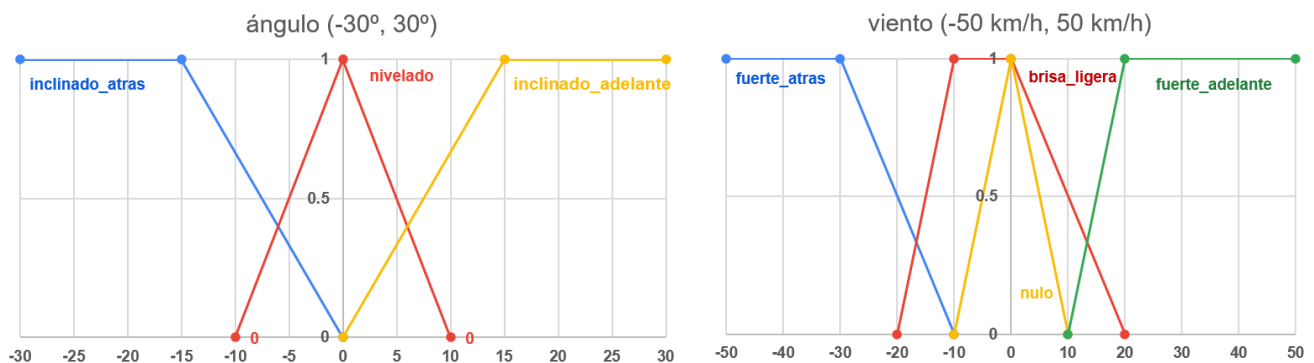
Queremos diseñar un sistema difuso para implementarlo en el dron que, en base al ángulo de inclinación del dron y a la velocidad del viento, sea capaz de ajustar tanto la potencia de sus motores como la posición de sus alerones para mantenerlo en una posición segura.



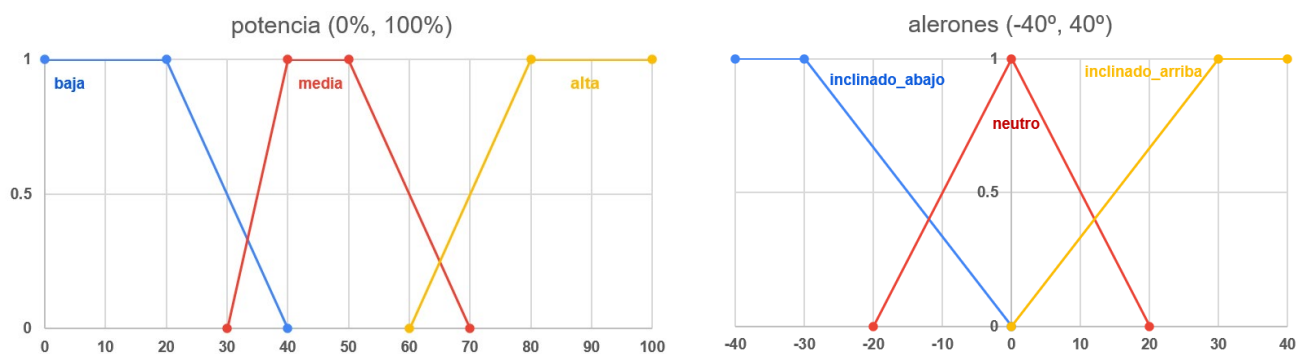
El dron dispone de los siguientes sensores:

- Sensor de inclinación que mide el ángulo de inclinación del dron sobre la horizontal. Este ángulo crisp se fusifica en la variable difusa “ángulo”, con los valores difusos: “inclinado_atras”, “nivelado”, e “inclinado_adelante”.
- Sensor de viento, que mide la velocidad del viento. Esta velocidad crisp se fusifica en la variable difusa “viento”, con los valores difusos: “fuerte_atras”, “brisa_ligera”, “nulo”, y “fuerte_adelante”.

Los grados de pertenencia de cada uno de estos valores (donde el universo de discurso son grados y km/h, respectivamente) son:



En función de estos dos sensores, el sistema debe calcular la potencia de los motores (variable “potencia”, con valores “baja”, “media” y “alta”) y la posición de los alerones (variable “alerones”, con valores “inclinado_abajo”, “neutro” e “inclinado_arriba”). Los grados de pertenencia de estos valores (el universo de discurso se mide en % y grados, respectivamente) son:



Los ingenieros de diseño han analizado distintas configuraciones y han definido la siguiente estrategia de control para actuar sobre el par “[potencia, alerones]” para conseguir una correcta estabilización:

angulo vs. viento	fuerte_atras	brisa_ligera	nulo	fuerte_adelante
inclinado_atras	[alta, inclinado_abajo]	[media, somewhat inclinado_abajo]	[baja, inclinado_arriba]	[more-or-less baja, neutro]
nivelado	[media, inclinado_abajo]	[somewhat media, neutro]	[media, somewhat neutro]	[alta, very inclinado_arriba]
inclinado_adelante	[very baja, neutro]	[baja, neutro]	[alta, very inclinado_abajo]	[extremely alta, inclinado_arriba]

Adicionalmente, los ingenieros han realizado pruebas con datos crisp del sensor de inclinación y el de viento, obteniendo los siguientes datos de telemetría. Por tanto, debemos modelar las reglas correspondientes:

Prueba	Sensor inclinación	Sensor viento	[potencia, alerones]
P1	-5	-30	[media, neutro]
P2	20	35	[alta, inclinado_arriba]
P3	-15	-40	[alta, inclinado_abajo]

Por ejemplo, en P1 con “Sensor inclinación = -5”, habrá que comprobar “(angulo inclinado_atras AND nivelado)”, mientras que con “Sensor viento = -30” habrá que comprobar simplemente “(viento fuerte_atras)”.

En el sistema Fuzzy Clips a desarrollar habrá que definir **cuatro variables difusas**: angulo, viento, potencia y alerones. Además, se definirá un **template/clase Dron** y una **única instancia de la misma**, que almacenará la siguiente información (inputs + outputs):

INPUTS:

- sensor-inclinacion-crisp del dron (en grados),
- sensor-viento-crisp del dron (en km/h),

OUTPUTS:

- mom-potencia-crisp: potencia de los motores (en %), decodificado por momentum,
- max-potencia-crisp: potencia de los motores (en %), decodificado por maximum,
- mom-alerones-crisp: posición de los alerones (en grados), decodificado por momentum,
- max-alerones-crisp: posición de los alerones (en grados), decodificado por maximum.

Nota. Se aconseja que todos los valores crisp sean del tipo FLOAT. Es recomendable no utilizar tildes en los nombres.

El **funcionamiento** será:

1. Se solicitarán los inputs al usuario como valores crisp (simulando el input real del sistema). Estos valores se almacenarán en la instancia actual.

2. Estos valores crisp se deberán fusificar, para convertir del valor crisp al valor fuzzy correspondiente. **Nota:** para fusificar los valores crisp se utilizará la función *fuzzify* dada en el apartado 2.3 y que proporciona Fuzzy-Clips. Utilizaremos siempre un delta = 0. **Hay que copiar dicha función a nuestro modelo antes de poder invocarla.**
3. Las variables “potencia” y “alerones” se calcularán en base al razonamiento difuso dado por las reglas del problema.
4. Tras finalizar todo el razonamiento difuso se deberán obtener los outputs, almacenarlos en la instancia actual y mostrarlos por pantalla. Para ello se defusificarán las variables “potencia” y “alerones” para obtener las salidas “XXX-potencia-crisp” y “XXX-alerones-crisp”. Se utilizarán las funciones “*moment-defuzzify*” y “*maximum-defuzzify*” para obtener las correspondientes variables “XXX-”. Estos valores serán los que finalmente utilizará el dron. Para finalizar el razonamiento y evitar bucles infinitos habrá que usar “(halt)” en la parte derecha de la regla que realice la defusificación. **MUY IMPORTANTE:** esta regla debe ser la **última** en ser ejecutada (uso de “salience”) y **SOLO** se debe ejecutar una vez.

Una vez implementado el sistema difuso, se deberán evaluar diversos escenarios, analizando y comparando los resultados crisp obtenidos.

Notas importantes:

- Dado que el esquema general de las reglas se repite, se recomienda diseñar una regla, probar su corrección y aplicarla en las restantes.
- Para modificar en una regla el valor de un slot de una instancia, que se ha instanciado previamente en la parte izquierda, se puede utilizar el comando modify (ver ejemplo en el anexo). **MUY IMPORTANTE:** no es lo mismo modificar el valor de un slot de una instancia existente con modify, que asertar una nueva instancia con el valor del slot modificado, que creará nuevas instancias. Aunque haya que modificar los valores de varios slots, **solo** utilizaremos un comando modify.

Como ejemplos de resultados en distintos escenarios tenemos los siguientes valores, según la defusificación por momentum (mom) o maximum (max). **Nota:** los valores se han **truncado** a dos decimales

	sensor_viento = -30	sensor_viento = 8	sensor_viento = 30
sensor_inclinación = -5	mom-potencia-crisp: 62.81 max-potencia-crisp: 47.5 mom-alerones-crisp: -14.48 max-alerones-crisp: -27.5	mom-potencia-crisp: 41.08 max-potencia-crisp: 48.73 mom-alerones-crisp: -3.39 max-alerones-crisp: 0.0	mom-potencia-crisp: 55.52 max-potencia-crisp: 85.0 mom-alerones-crisp: 14.40 max-alerones-crisp: 30.6
sensor_inclinación = 20	mom-potencia-crisp: 13.77 max-potencia-crisp: 10.0 mom-alerones-crisp: -1.42E-15 max-alerones-crisp: 0.0	mom-potencia-crisp: 40.36 max-potencia-crisp: 47.0 mom-alerones-crisp: -10.80 max-alerones-crisp: -25.42	mom-potencia-crisp: 84.44 max-potencia-crisp: 90.0 mom-alerones-crisp: 26.0 max-alerones-crisp: 35.0

Estos valores se han conseguido realizando una fusificación en forma de valores singleton. Es decir, cuando se ha utilizado la función *fuzzify* se ha hecho con un delta=0, ej. “(fuzzify variable-difusa valor 0)”.

NOTA IMPORTANTE. Es imprescindible utilizar correctamente las funciones: 1) (**clear**), que limpia/vacía la base de hechos, 2) (**reset**), que inicializa la base de hechos, junto con la instancia a utilizar, y determina las reglas iniciales a aplicar, y 3) (**run**), que comienza la ejecución de las reglas.

AMPLIACIÓN VOLUNTARIA (NO SE NECESITA PARA LA EVALUACIÓN DE LA PRÁCTICA). Se desea simular un botón ECO (con valores ON/OFF) que deberá solicitarse también como entrada. Si el valor es ON, entonces se deberá modificar “potencia”, asertando el hecho “baja” para ahorrar energía.

Por ejemplo, algunos resultados cuando el botón ECO está a ON son:

	sensor_viento = 8	sensor_viento = 30
sensor_inclinación = -5	mom-potencia-crisp: 28.32 max-potencia-crisp: 10.0 mom-alerones-crisp: -3.39 max-alerones-crisp: 0.0	mom-potencia-crisp: 40.12 max-potencia-crisp: 10.0 mom-alerones-crisp: 14.40 max-alerones-crisp: 30.6
sensor_inclinación = 20	mom-potencia-crisp: 28.78 max-potencia-crisp: 10.0 mom-alerones-crisp: -6.01 max-alerones-crisp: 0.0	mom-potencia-crisp: 50.0 max-potencia-crisp: 50.0 mom-alerones-crisp: 26.0 max-alerones-crisp: 35.0